COS 30049 Computing Technology Innovation Project

Week5: Data Pre-Processing in Machine Learning via Python

Lecture2

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture, and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures, and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.



Objectives:

1. Understand and learn to use Pandas Library
2. Understand and learn to use Scikit-learn Library
3. Understand and learn to use Matplotlib Library
4. Understand and learn to use Seaborn Library
5. Assignment 2

These three libraries are the ones named in Assignment 2 (A2)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Python Installation Guide

Before we start, make sure you have correctly installed python:

Comprehensive Guide_ Installing Python, Setting Up VS Code, and Configuring the System Environment Path.pdf

(Optional) For Students Who Want to Control Their Python Environment: Python Conda Environment Setup:

Python Environment Setup.pdf

We have provided comprehensive instructions to help students install Python and the necessary tools for their projects. For detailed steps, please refer to Module Week 5.

Optional: Install Conda for Python Environment Management
What is Conda?
Conda is a package management tool from Anaconda, designed to simplify the installation, management, and updating of Python packages and other software. It also facilitates the creation of virtual environments, enabling you to isolate project dependencies, ensuring compatibility and avoiding conflicts. Conda is cross-platform, making it an ideal choice for managing Python environments efficiently.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Pandas (A2)



pandas - Python Data Analysis Library (pydata.org)

Pandas is a powerful Python library for data manipulation and analysis. It provides efficient and convenient data structures and data analysis tools, widely used in data cleaning, data transformation, data visualization, and machine learning.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Pandas (A2)

1. Prepare environment

You can install pandas with:

(x.x.x is the *pandas* version of you want, not necessary.)

```
pip install pandas
pip install pandas==x.x.x
```

or

```
conda install pandas
conda install pandas=1.3.3
```

If you need to check the version of pandas:

```
import pandas as pd
print(pd.__version__)
```

or

```
pip show pandas
```



If you need to uninstall pandas:

```
pip uninstall pandas
```

or

```
conda remove pandas
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

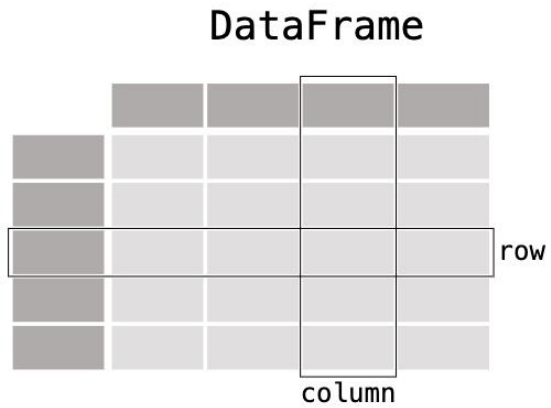
Pandas (A2)

2. Data Structure

In pandas, a data table is called a **DataFrame**.

A two-dimensional table-like data structure, similar to a spreadsheet or SQL data table, composed of multiple Series and capable of storing various data types.

```
df = pd.DataFrame(data)
```



SWIN BUR NE
SWINBURNE UNIVERSITY OF TECHNOLOGY

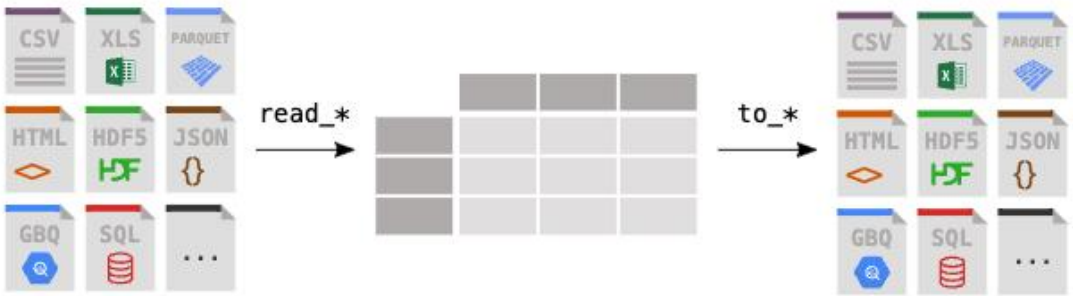
Pandas (A2)

3. Data Read & Write

```
titanic = pd.read_csv("data/titanic.csv")
```

```
titanic = pd.read_excel("titanic.xlsx", sheet_name="passengers")
```

```
titanic.to_excel("titanic.xlsx", sheet_name="passengers", index=False)
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Pandas (A2)

4. Data Cleaning and Processing

Handling Missing Data: Provides methods to fill, delete, and interpolate missing data.

Data Transformation: Supports data type conversion, data sorting, data reshaping, and other operations.

```
type(titanic["Age"])
```

→ `pandas.core.series.Series`

pandas: A Python library for data manipulation and analysis.
core.series: Indicates the Series class within the core module of Pandas.
Series: A one-dimensional array-like data structure in Pandas, similar to a table column, capable of holding various data types.

```
age_sex = titanic[["Age", "Sex"]]
age_sex.head()
```

→

	Age	Sex
0	22.0	male
1	38.0	female
2	26.0	female
3	35.0	female
4	35.0	male

```
titanic["Age"].shape
```

→ `(891,)`

This indicates that the "Age" column contains 891 entries or rows. The comma indicates that it's a one-dimensional structure, specifically a Series with 891 elements.

```
class_23 = titanic[titanic["Pclass"].isin([2, 3])]
class_23.head()
```

↓

	PassengerId	Survived	Pclass	...	Fare	Cabin	Embarked
0	1	0	3	...	7.2500	NaN	S
2	3	1	3	...	7.9250	NaN	S
4	5	0	3	...	8.0500	NaN	S
5	6	0	3	...	8.4583	NaN	Q
7	8	0	3	...	21.0750	NaN	S

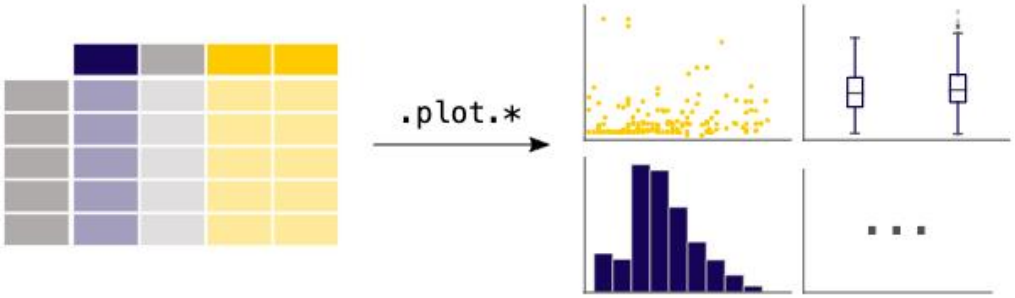
[5 rows x 12 columns]

SWINBURNE UNIVERSITY OF TECHNOLOGY

Pandas (A2)

5. Data Visualization

Integration with Matplotlib and Seaborn: Supports direct plotting of charts by integrating with visualization libraries like Matplotlib and Seaborn.



SWINBURNE UNIVERSITY OF TECHNOLOGY

Scikit-learn (sklearn) (A2)

Scikit-Learn, commonly known as sklearn

is a powerful Python library specifically designed for **machine learning and data analysis**. It is built on top of SciPy and provides simple and efficient **tools** for data mining and data analysis.



scikit-learn: machine learning in Python — scikit-learn 1.5.1 documentation

You can install scikit-learn (sklearn) with:

```
pip install scikit-learn
```

or

```
conda install scikit-learn
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Scikit-learn (sklearn) (A2)

1. A variety of machine learning algorithms

Provides a large number of machine learning algorithms, including classification, regression, clustering, and dimensionality reduction.

Common algorithms: linear regression, logistic regression, support vector machines (SVM), decision trees, random forests, K-means clustering, etc.



SWINBURNE UNIVERSITY OF TECHNOLOGY

Scikit-learn (sklearn) (A2)

2. Dataset Examples

Provides some datasets which can be easily used for machine learning practice, including classification, regression, clustering, and dimensionality reduction.

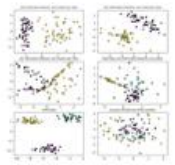
```
from sklearn import

datasets

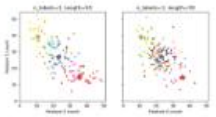
iris = datasets.load_iris()
```

Dataset examples

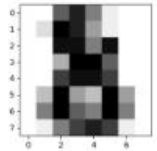
Examples concerning the `sklearn.datasets` module.



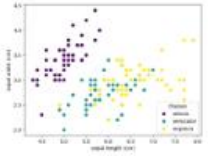
Plot randomly generated classification dataset



Plot randomly generated multilabel dataset



The Digit Dataset



The Iris Dataset

`fetch_california_housing`	Load the California housing dataset (regression).
`fetch_covtype`	Load the covertype dataset (classification).
`fetch_kddcup99`	Load the kddcup99 dataset (classification).
`fetch_lfw_pairs`	Load the Labeled Faces in the Wild (LFW) pairs dataset (classification).
`fetch_lfw_people`	Load the Labeled Faces in the Wild (LFW) people dataset (classification).
`fetch_olivetti_faces`	Load the Olivetti faces data-set from AT&T (classification).
`fetch_openml`	Fetch dataset from openml by name or dataset id.
`fetch_rcv1`	Load the RCV1 multilabel dataset (classification).
`fetch_species_distributions`	Loader for species distribution dataset from Phillips et.
`get_data_home`	Return the path of the scikit-learn data directory.
`load_breast_cancer`	Load and return the breast cancer wisconsin dataset (classification).
`load_diabetes`	Load and return the diabetes dataset (regression).
`load_digits`	Load and return the digits dataset (classification).

Scikit-learn (sklearn) (A2)

3. Dataset Splitting

In machine learning, splitting a dataset into training and testing subsets is a crucial step. The training set is used to train the model, and the testing set is used to evaluate its performance on unseen data. The train_test_split function is commonly used for this purpose.

```
sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=
None, shuffle=True, stratify=None)
```

arrays: Input arrays that you want to split. These could be features (X) and target labels (y).
test_size: Proportion of the dataset to include in the test split. It can be a float (e.g., 0.33 means 33% of the data will be used for testing).
train_size: Proportion of the dataset to include in the train split (often left as None since it can be inferred from test_size).
random_state: Controls the shuffling applied to the data before splitting. Passing an integer makes the splitting reproducible.
shuffle: Whether to shuffle the data before splitting. By default, it is True.
stratify: If not None, data is split in a stratified fashion, using this as the class labels.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Scikit-learn (sklearn) (A2)

3. Dataset Splitting

Examples:

sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=

None, shuffle=True, stratify=None)

```
>>> import numpy as np
>>> from sklearn.model_selection import train_test_split
>>> X, y = np.arange(10).reshape((5, 2)), range(5)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
>>> list(y)
[0, 1, 2, 3, 4]
```

→

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.33, random_state=42)
...
>>> X_train
array([[4, 5],
       [0, 1],
       [6, 7]])
>>> y_train
[2, 0, 3]
>>> X_test
array([[2, 3],
       [8, 9]])
>>> y_test
[1, 4]
```

→

```
>>> train_test_split(y, shuffle=False)
[[0, 1, 2], [3, 4]]
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Matplotlib (A2)

Matplotlib is a comprehensive library for **creating static, animated, and interactive visualizations** in Python. Matplotlib makes easy things easy and hard things possible.



https://matplotlib.org/

You can install matplotlib with:

```
pip install  matplotlib
```

or

```
conda install matplotlib
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Matplotlib (A2)

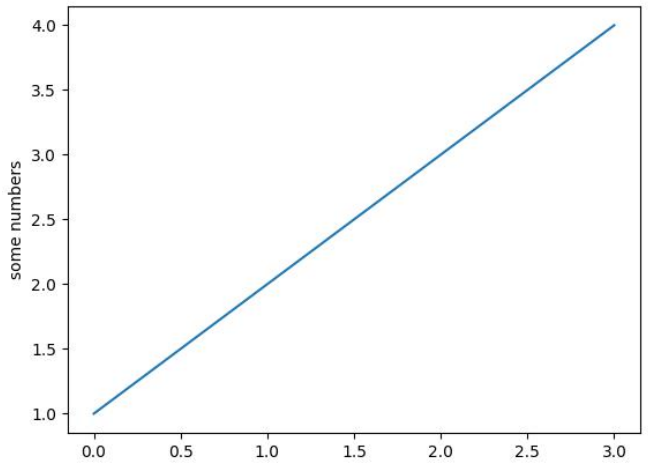
Pyplot

matplotlib.pyplot is a module in the Matplotlib library that provides a set of functions to create and customize plots in Python. It allows you to build visualizations step by step, with each function modifying the plot in some way.

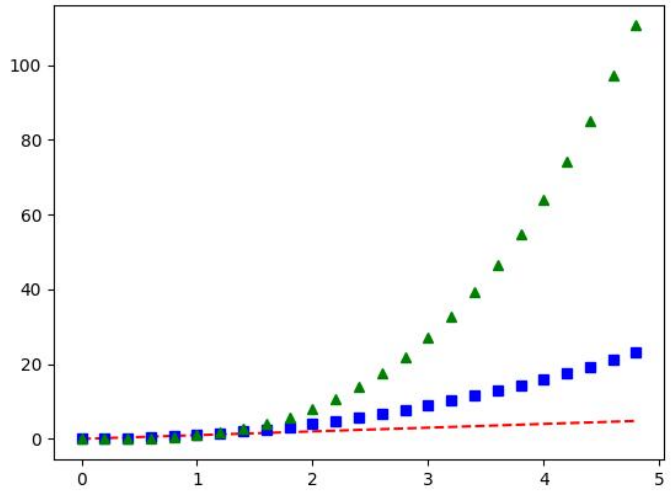
Each pyplot function makes some change to a figure:

e.g., creates a figure, creates a plotting area in a figure, plots some lines in a plotting area, decorates the plot with labels, etc.

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4])
plt.ylabel('some numbers')
plt.show()
```




```
import numpy as np # evenly sampled time at 200ms intervals
t = np.arange(0., 5., 0.2) # red dashes, blue squares and green triangles
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')
plt.show()
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Matplotlib (A2)

Pyplot

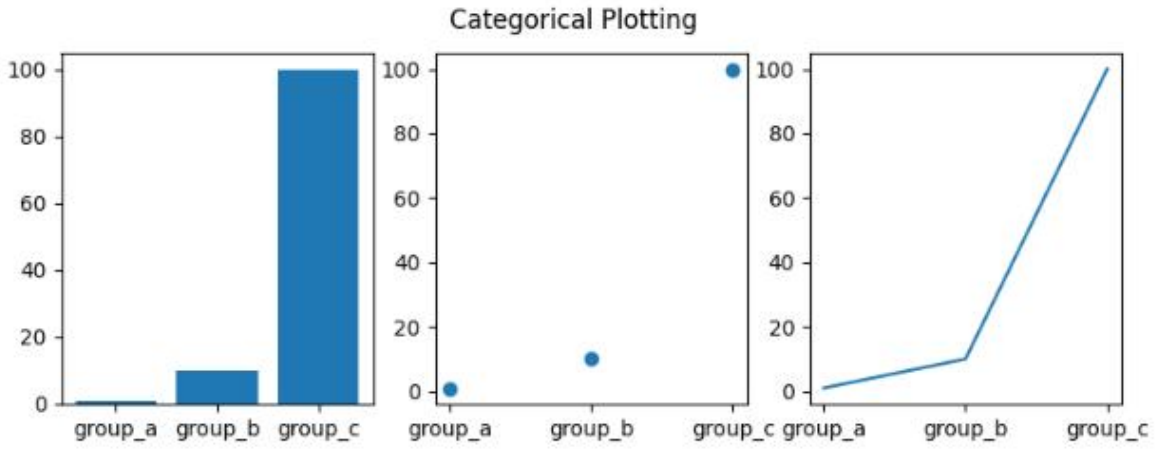
Create a figure with three subplots, each displaying the same categorical data using different types of plots. The data consists of three groups (group_a, group_b, group_c) with corresponding values (1, 10, 100).

Analyze the code according to the plotting and tell the functions of different code lines.

```
names = ['group_a', 'group_b', 'group_c']
values = [1, 10, 100]

plt.figure(figsize=(9, 3))

plt.subplot(131)
plt.bar(names, values)
plt.subplot(132)
plt.scatter(names, values)
plt.subplot(133)
plt.plot(names, values)
plt.suptitle('Categorical Plotting')
plt.show()
```



bar plot. scatter plot. line plot.

Matplotlib (A2)

Pyplot

```python
import matplotlib.pyplot as plt

# Define the categories and their corresponding values
names = ['group_a', 'group_b', 'group_c']
values = [1, 10, 100]

# Create a figure with a specific size
plt.figure(figsize=(9, 3))

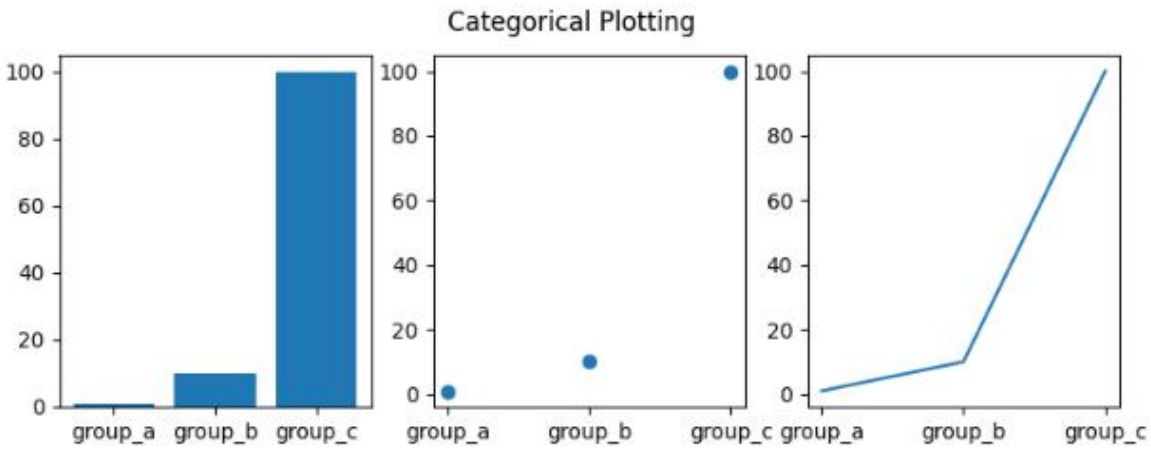
# Create the first subplot (Bar Plot)
plt.subplot(131)
plt.bar(names, values)

# Create the second subplot (Scatter Plot)
plt.subplot(132)
plt.scatter(names, values)

# Create the third subplot (Line Plot)
plt.subplot(133)
plt.plot(names, values)

# Set the overall title for the figure
plt.suptitle('Categorical Plotting')

# Display the figure
plt.show()
```



bar plot. scatter plot. line plot.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Matplotlib (A2)

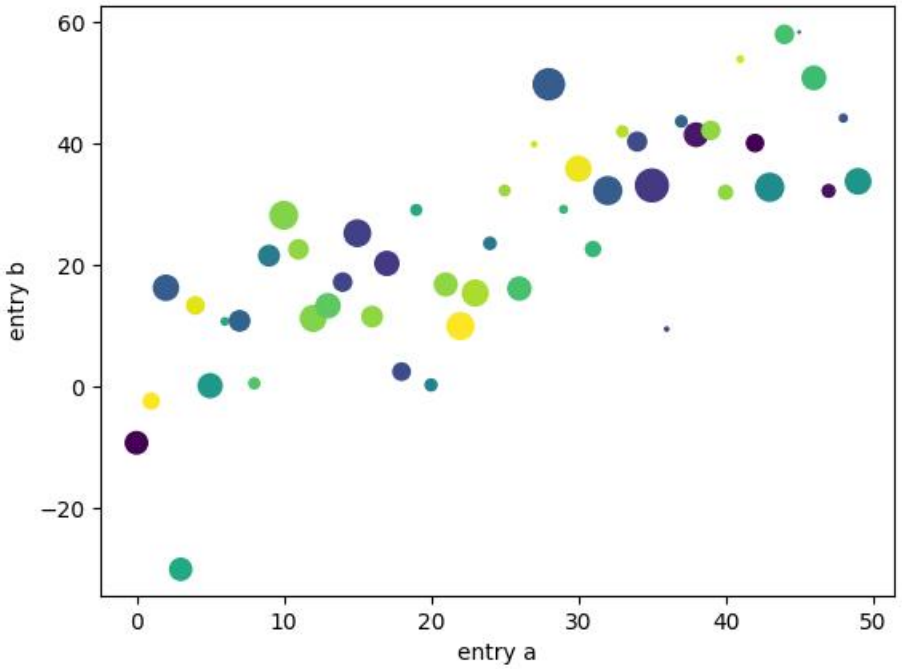
Pyplot

```
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

plt.scatter('a', 'b', c='c', s='d', data=data)
plt.xlabel('entry a')
plt.ylabel('entry b')
plt.show()
```

This code generates a scatter plot with bubble sizes and colors representing additional dimensions of the data. The x-axis (a) corresponds to a sequence of numbers, while the y-axis (b) is a combination of a with some added noise. The color (c) and size (s) of each bubble are determined by random values.

Analyze the code according to the plotting and tell the functions of different code lines.



SWINBURNE UNIVERSITY OF TECHNOLOGY

Matplotlib (A2)

Pyplot

```python
import numpy as np
import matplotlib.pyplot as plt

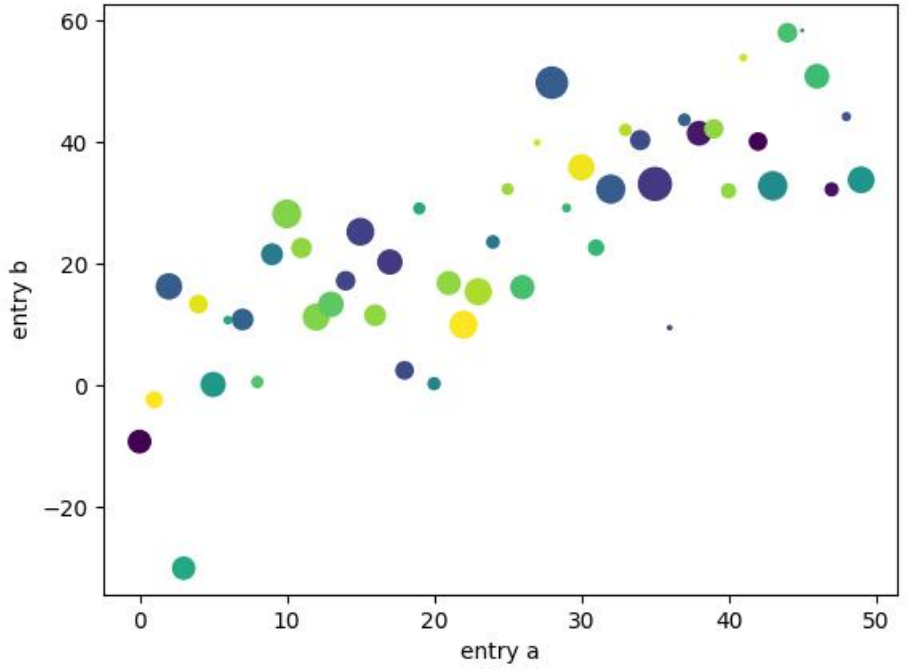
# Generate data for the plot
data = {
    'a': np.arange(50),  # x-axis values: a sequence of numbers from 0 to 49
    'c': np.random.randint(0, 50, 50),  # Random colors for each point
    'd': np.random.randn(50)  # Random sizes for each point
}

# Generate y-axis values with added noise and calculate bubble sizes
data['b'] = data['a'] + 10 * np.random.randn(50)  # y-axis values: 'a' values with added noise
data['d'] = np.abs(data['d']) * 100  # Bubble sizes: based on random values scaled up

# Create a scatter plot with varying bubble sizes and colors
plt.scatter('a', 'b', c='c', s='d', data=data)

# Label the x and y axes
plt.xlabel('entry a')
plt.ylabel('entry b')

# Display the plot
plt.show()
```



This code generates a scatter plot where:

The x-axis represents sequential data from 0 to 49.

The y-axis is a noisy version of the x-axis values.

The color and size of each point are determined by randomly generated data.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Seaborn (A2)

Difference between matplotlib and seaborn

- Matplotlib is the most common library in Python, offering a wide range of plotting options and customization features, allowing the creation of complex and highly customized visualizations.
- Seaborn is a higher-level visualization library built on top of Matplotlib, focusing on the aesthetic design of data visualizations and the creation of statistical graphics. Seaborn emphasizes data exploration and analysis, with a strong focus on aesthetic styles. However, Seaborn's underlying functionality is based on Matplotlib.

- Matplotlib requires a lot of code to create detailed plots, while Seaborn offers a higher-level, more aesthetically pleasing interface, though at the cost of customization:
- Seaborn provides "shortcuts" by encapsulating commonly used visualization processes into functions, making the code more concise and allowing for clear and attractive visual outputs with just one line of code. The main drawback is its limited customization capabilities, as it mainly supports predefined visualization templates.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Seaborn (A2)

Seaborn is a Python data **visualization** library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.



https://seaborn.pydata.org/

You can install seaborn with:

```
pip install  seaborn
```

or

```
conda install seaborn
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Seaborn (A2)

This code generates a scatter plot using the tips dataset, visualizing the relationship between the total bill and the tip amount, with points colored based on whether the customer was a smoker or not.

1.Imports and Theme Setup:

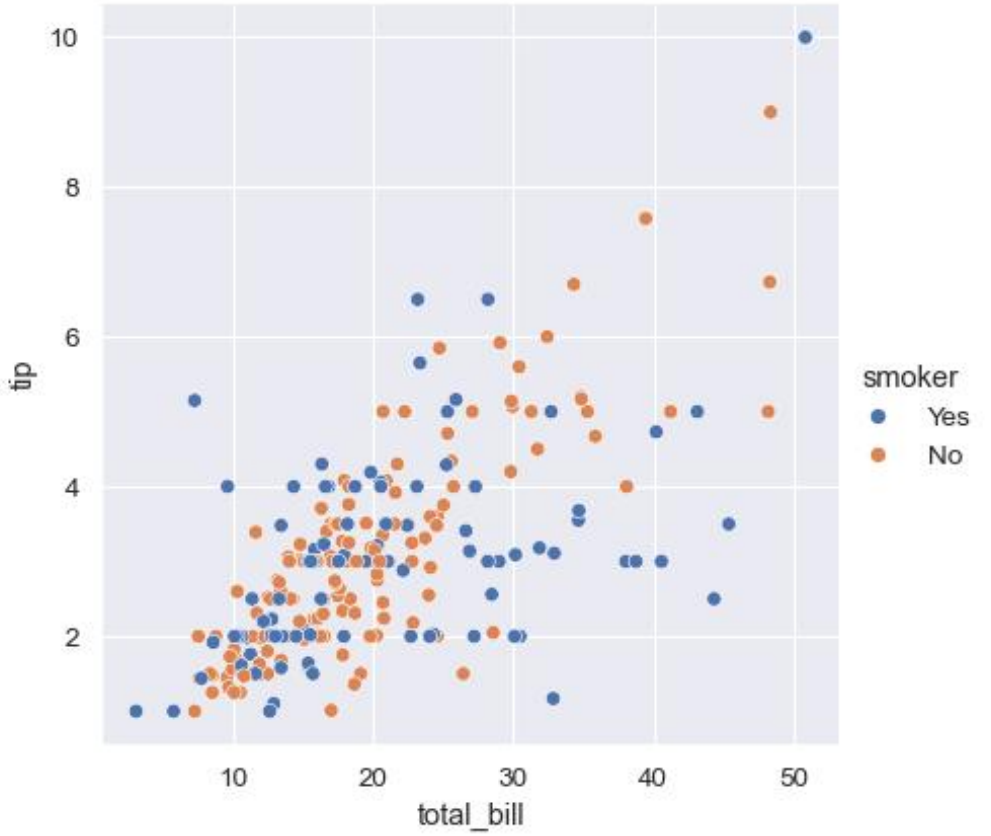
1. Necessary libraries (numpy, pandas, matplotlib, seaborn) are imported.
2. The plot theme is set to darkgrid for a clean and readable background.

2.Scatter Plot Creation:

1. A scatter plot is generated using total_bill on the x-axis and tip on the y-axis.
2. Points are colored by the smoker status (Yes/No).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="darkgrid")
```

```
sns.relplot(data=tips, x="total_bill", y="tip", hue="smoker")
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Seaborn (A2)

Use the Seaborn library to create line plots for two different datasets, visualizing relationships between variables over time.

```
dowjones = sns.load_dataset("dowjones")
sns.relplot(data=dowjones, x="Date", y="Price", kind="line")
```

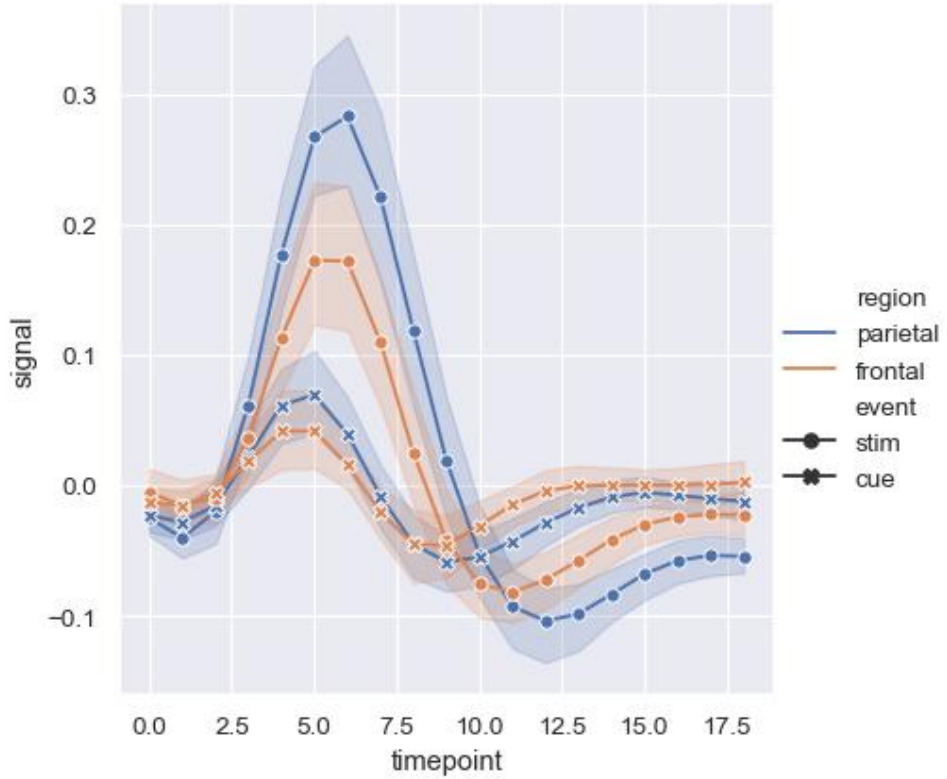
```
sns.relplot(
    data=fmri, kind="line",
    x="timepoint", y="signal", hue="region", style="event",
    dashes=False, markers=True,
)
```

1.First Plot (Dow Jones):

1. The dowjones dataset is loaded.
2. A line plot is created showing the relationship between Date and Price.

2.Second Plot (fMRI Data):

1. The fmri dataset is used.
2. A line plot is generated showing signal over timepoint, differentiated by region (hue) and event (style).
3. Markers are enabled, and dashes are disabled for the lines.



SWINBURNE UNIVERSITY OF TECHNOLOGY

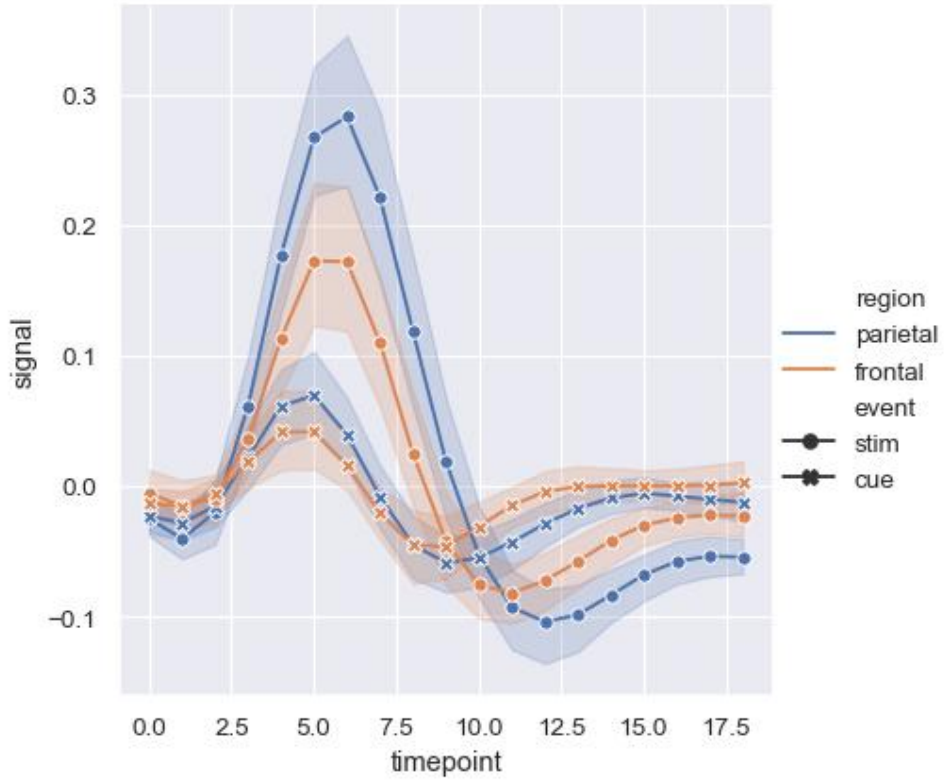
Seaborn (A2)

```python
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dowjones dataset and create a line plot
dowjones = sns.load_dataset("dowjones")
sns.relplot(data=dowjones, x="Date", y="Price", kind="line")

# Create a line plot using the fmri dataset with additional styling
sns.relplot(
    data=sns.load_dataset("fmri"), kind="line",
    x="timepoint", y="signal", hue="region", style="event",
    dashes=False, markers=True,
)

plt.show()
```



This code generates two distinct line plots:
one visualizing stock prices over time
and another visualizing brain activity signals over time, segmented by regions and events.

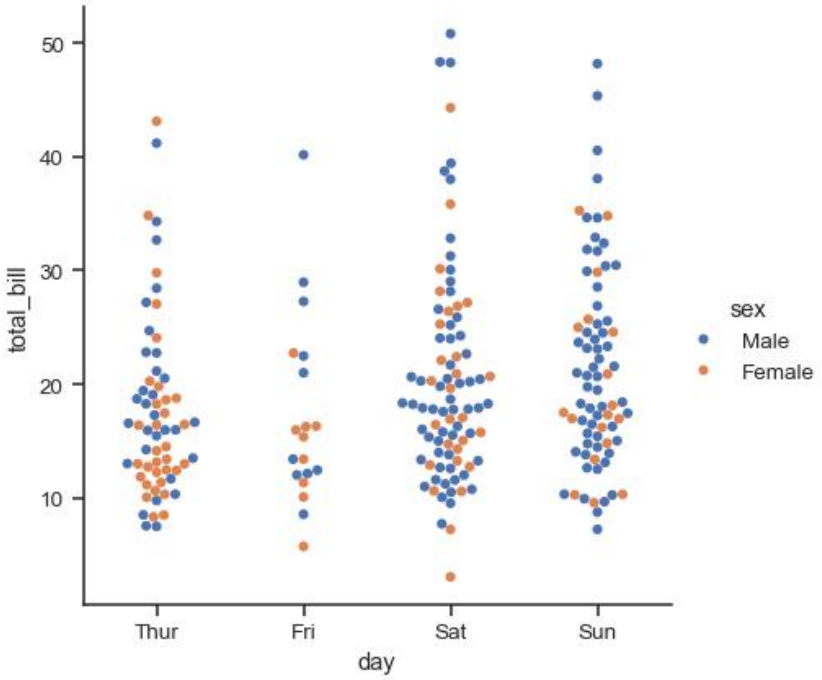
SWINBURNE UNIVERSITY OF TECHNOLOGY

Seaborn (A2)

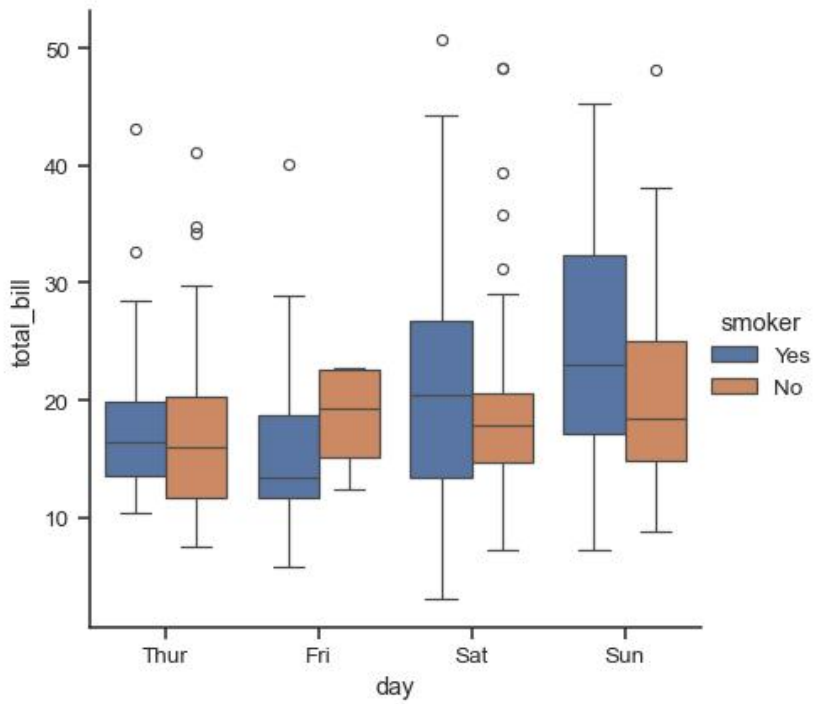
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="darkgrid")
```

```
tips = sns.load_dataset("tips")
```

```
sns.catplot(data=tips, x="day", y="total_bill", hue="sex", kind="swarm")
```




```
sns.catplot(data=tips, x="day", y="total_bill", hue="smoker", kind="box")
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Thank you

SWINBURNE UNIVERSITY OF TECHNOLOGY